

Honeypot & Attack Dashboard — Mini SOC Project

Student project · RHEL system administration & security monitoring

Goal: Build an isolated "honeypot" server that intentionally attracts and records intrusion attempts, then build a live dashboard that visualizes who is attacking it, how, and how often — a small-scale version of what a real Security Operations Center (SOC) analyst monitors every day.

Safety note: The honeypot must run in an isolated lab network (host-only virtual network or a segregated VLAN), never directly exposed to the real internet on a home/school network without authorization. The goal is to observe and log simulated or lab-internal attack traffic safely, not to bait real internet attackers without proper isolation and permission.

Environment

Two RHEL VMs on an isolated host-only virtual network:

VM	Role
vm-honeypot	The exposed, deliberately weak target — SSH and a fake web login page
vm-attacker	A second VM used to simulate attacks against the honeypot (brute-force login attempts, port scans)

1. Build the Honeypot Target

- Install OpenSSH and a basic Nginx/PHP fake "admin login" page on `vm-honeypot` .
- Create a few decoy user accounts with weak, guessable passwords (lab-only — never reuse real credentials).
- Enable full logging: `auditd` for authentication events, Nginx access logs for the fake web login attempts.

Deliverable: honeypot service configs and a description of what "bait" you set up and why.

2. Simulate Attacks (from your own attacker VM)

- From `vm-attacker` , run a simple brute-force script against SSH (e.g. a small Bash/Python loop trying a password list) and against the fake web login form.
- Run a basic port scan (`nmap`) against the honeypot to generate reconnaissance-style log entries.
- Vary attempts over time so your dashboard has a realistic pattern to show (bursts, repeated IPs, different usernames tried).

Deliverable: the attack script(s) used and a short log excerpt showing captured attempts.

3. Parse and Aggregate the Logs

- Write a script (Bash + `awk / grep` , or Python) that parses `/var/log/secure` (SSH attempts) and the Nginx access log, extracting: source IP, timestamp, username tried, and result (success/failure).
- Aggregate into a summary: top attacking IPs, most-tried usernames, attempts per hour.
- Store results in a simple structured format (CSV or SQLite) so the dashboard can read it.

Deliverable: the parsing script and one sample aggregated output file.

4. Build the Live Dashboard

- Build a lightweight web page (plain HTML/JS, or a small Flask/Python app) that reads the aggregated data and displays:
 - A running counter of total attack attempts
 - A table or chart of top attacker IPs
 - A live/refreshing feed of the most recent attempts
- Serve the dashboard with Nginx (reuse your earlier Nginx skills) on a separate port from the honeypot bait itself.
- Auto-refresh the dashboard every few seconds (simple JS `setInterval` + fetch, or a page meta-refresh) so it feels "live."

Deliverable: dashboard code + a screenshot of it showing real captured attack data.

5. Automate the Pipeline

- Wrap the log-parsing step in a systemd timer so the dashboard's data updates automatically (e.g. every minute) without manual re-runs.
- Optionally: trigger an alert (email, or a local desktop notification / webhook) when attempts from a single IP exceed a threshold in a short window — a basic intrusion detection rule.

Deliverable: the systemd service/timer files and (if implemented) the alert rule.

6. Harden and Reflect

After capturing enough attack data, apply real defenses and re-test:

- Enable fail2ban to auto-ban IPs after repeated failures — rerun your attack script and show the difference in the logs/dashboard.
- Write a short reflection: which attacks were easiest to detect, which would be hardest to catch in a real environment, and what you'd add with more time (e.g. GeoIP lookup, Slack alerts, a real SIEM like Wazuh).

Suggested Timeline

Day	Focus
1	Set up isolated network + honeypot VM with bait services
2	Build and run attack simulation scripts from the attacker VM
3	Write log-parsing/aggregation script
4	Build the live dashboard
5	Automate with systemd timer, add fail2ban, write reflection

Skills Demonstrated

- Service configuration (SSH, Nginx, PHP)
- Log analysis and text processing (awk, grep, regex)
- Scripting (Bash and/or Python)
- systemd service/timer automation
- Basic security tooling (fail2ban, auditd, nmap)
- Simple web development for the dashboard
- Network isolation and lab safety practices

Why this project: *it turns abstract log files into something visible and immediate — watching your own dashboard light up in real time as a simulated attacker probes your machine makes intrusion detection concepts concrete in a way that reading logs alone never does, and it mirrors real SOC/security-analyst work far more closely than a standard web-hosting exercise.*